Programming with C++

COMP2011: Introduction

Gary Chan
James Cheung
Cindy Li

Department of Computer Science & Engineering
The Hong Kong University of Science and Technology
Hong Kong SAR, China

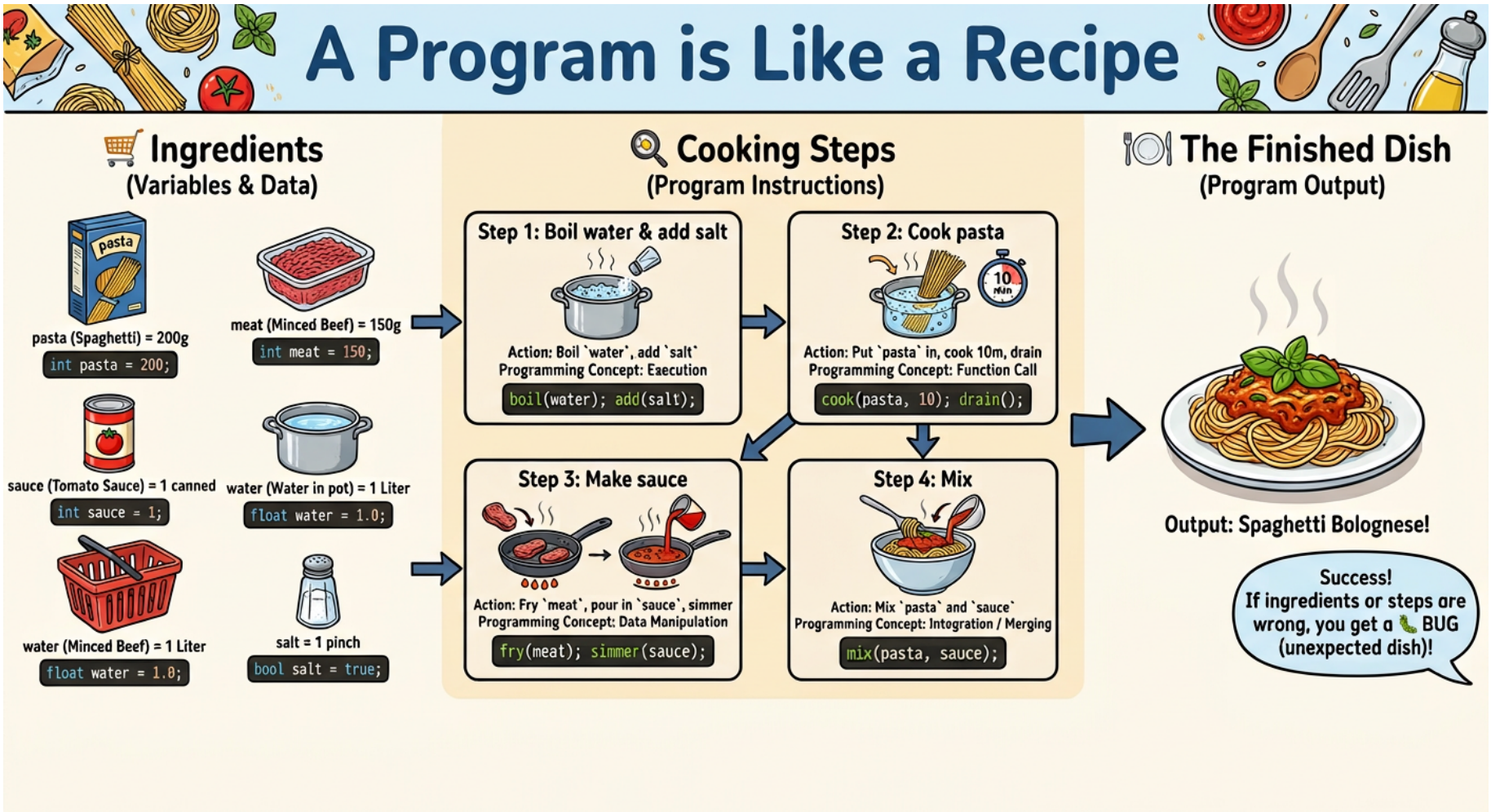


Course Objectives

- To learn how to solve problems by writing computer programs.
- To learn how to design a computer program.
- To learn how to program in C++.
- To learn how to debug a computer program.
- To learn object-oriented programming.
- To prepare you for COMP2012 (OOP & Data Structures), etc.

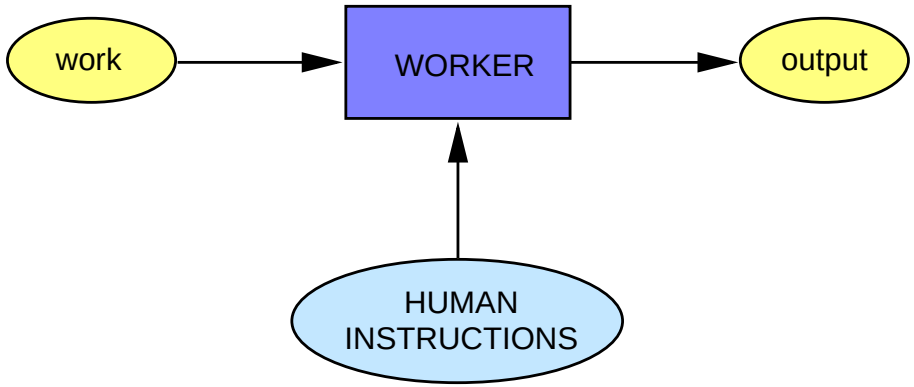
Question: *computer science* = *programming*?

What's a Computer Program?

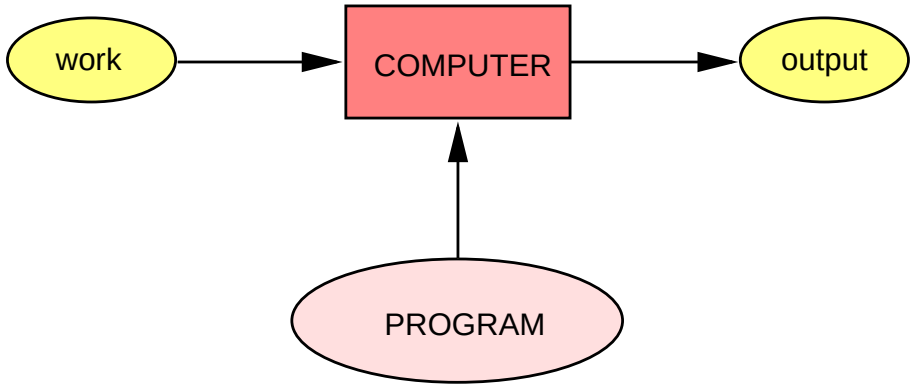


What's a Computer Program? ..

Human work model

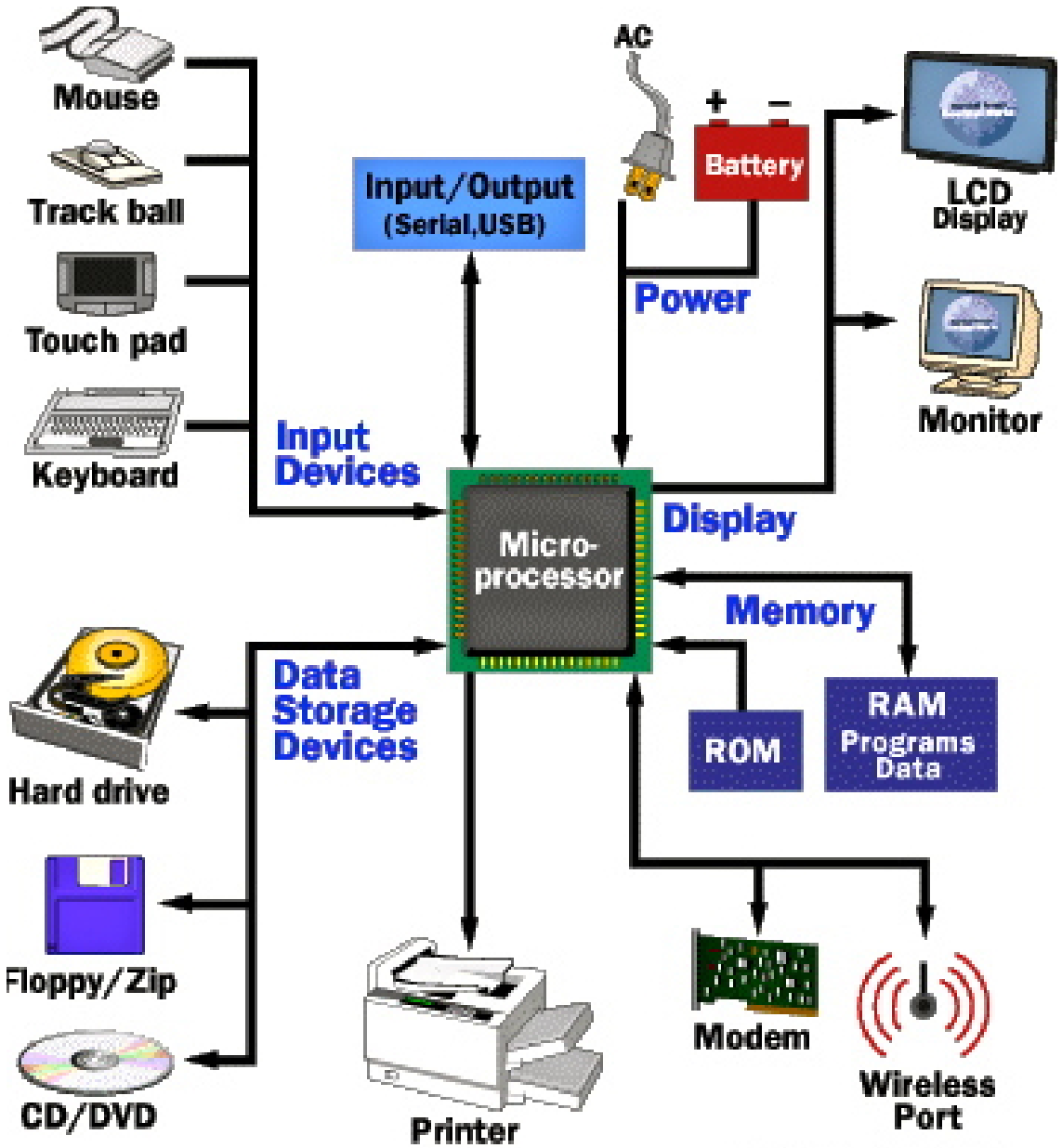


Computer work model



- A computer program is a set of machine-readable instructions that tells a computer how to perform a specific task. (During the execution of the program, it may interact with the users and its environment.)

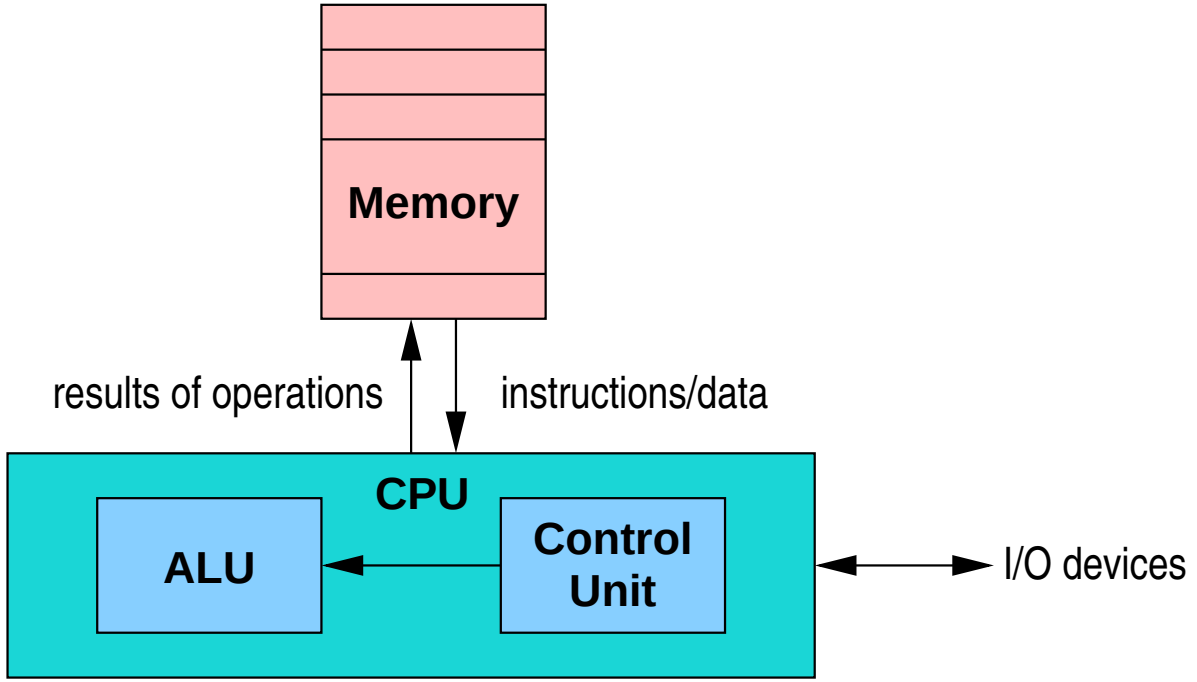
Schematic Diagram of a Personal Computer



von Neumann Computer Architecture

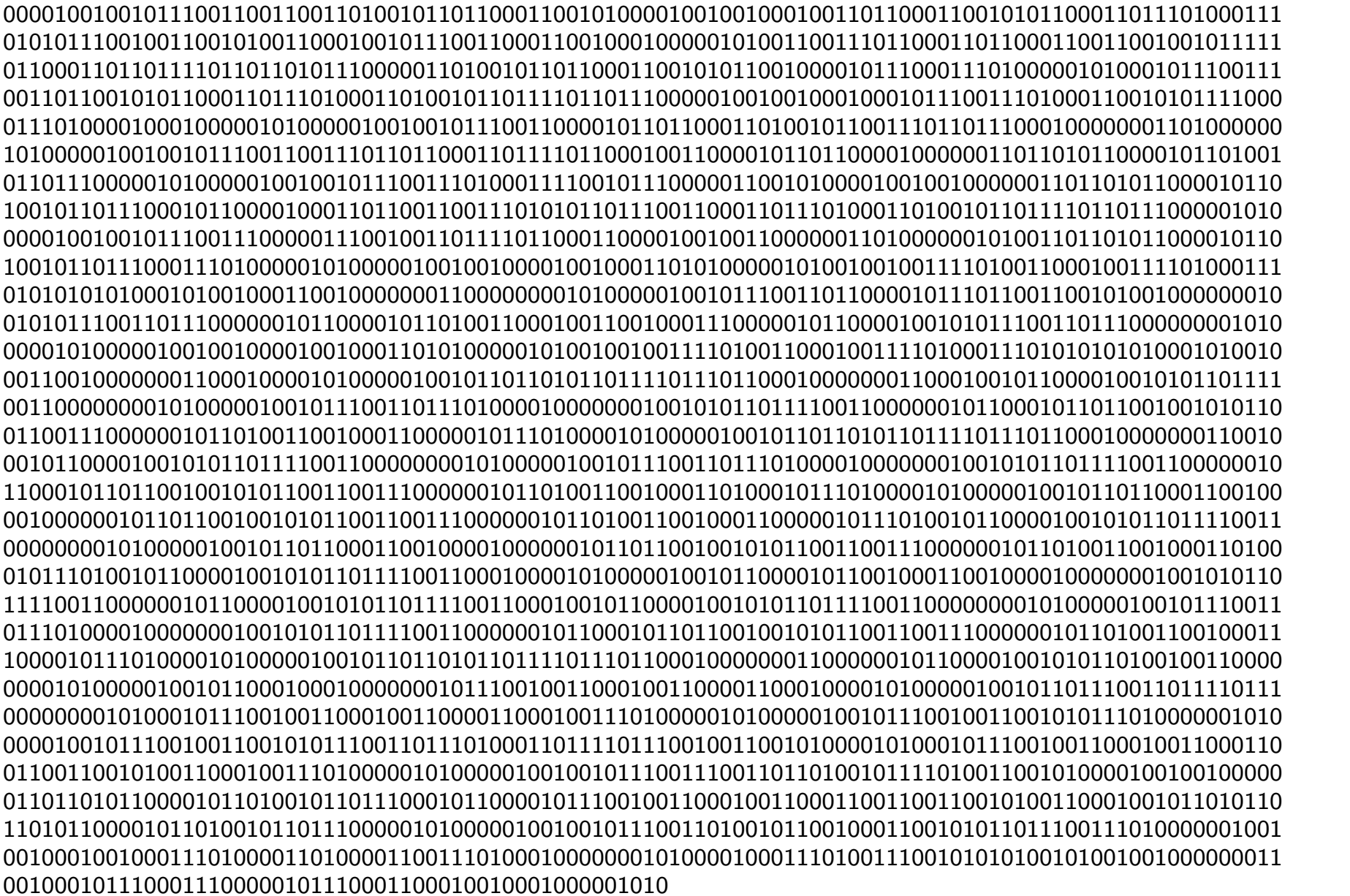






- Designed by John von Neumann, a mathematician, in 1945.
- It is still today's dominant computer architecture.
- CPU = Central Processing Unit
- ALU = Arithmetic Logic Unit.
- For efficiency, many programming languages, including C++, are designed to take advantage of the architecture.
- More on this in COMP2611 (Computer Organization).

Can You Understand This?



How About This?

```
main:
        !#PROLOGUE# 0
        save %sp,-128,%sp

        !#PROLOGUE# 1
        mov 1,%o0
        st %o0,[%fp-20]
        mov 2,%o0
        st %o0,[%fp-24]
        ld [%fp-20],%o0
        ld [%fp-24],%o1
        add %o0,%o1,%o0
        st %o0,[%fp-28]
        mov 0,%i0
        nop
```

Is This Better Now?

```
int main( )
{
    int x, y, z;

    x = 1;
    y = 2;
    z = x+y;

    return 0;
}
```

Example: Write a Program to Sum 2 Numbers

- There are 3 integer-value-holding objects: x, y, and z.
- x and y have the value of 1 and 2 respectively.
- z's value is the sum of x's and y's.

```
int main( )
{
    int x, y, z;

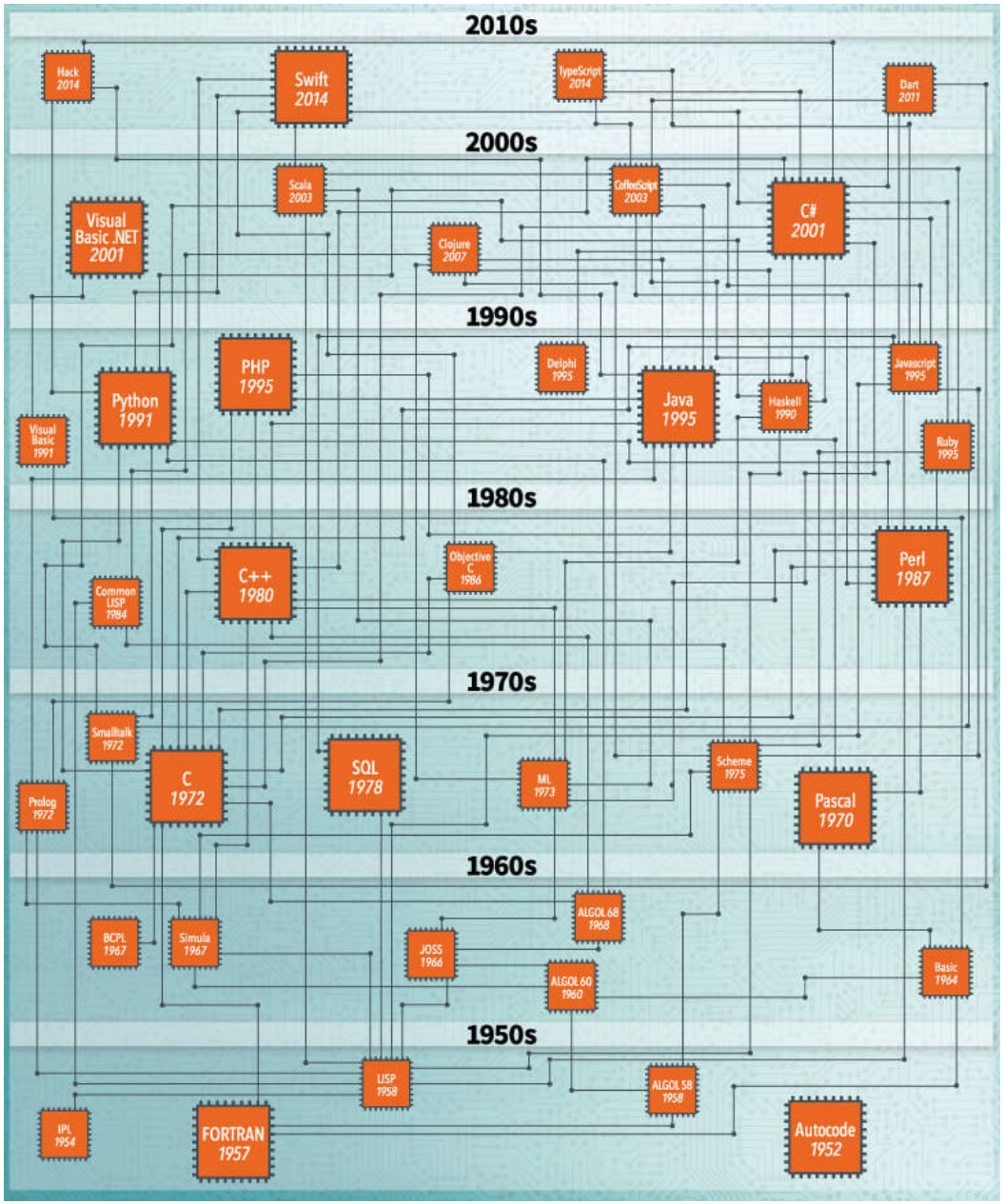
    x = 1;
    y = 2;
    z = x+y;

    return 0;
}
```

Levels of Programming Languages

- machine (binary) language is unintelligible
- assembly language is low level
  - mnemonic names for machine operations
  - explicit manipulation of memory addresses/contents
  - machine-dependent
- high level language
  - readable
    - instructions are easy to remember
    - faster coding
    - less error-prone (fewer bugs?)
    - easier to maintain
  - no mention of memory locations
  - machine-independent = portable

Chronology of Some Programming Languages



More details in this computer history website, and the following infographics.

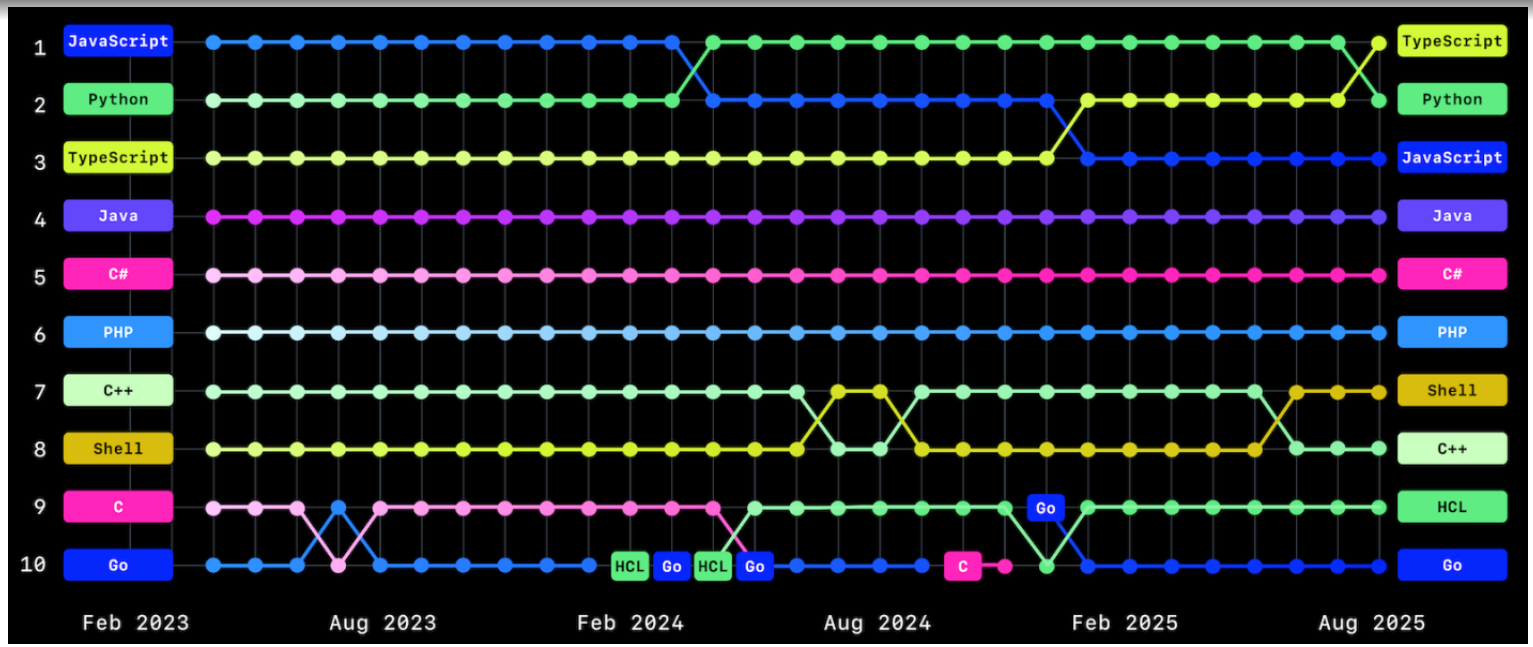
PYPL PopularitY: Most Popular Programming Languages

Worldwide, Aug 2026 :

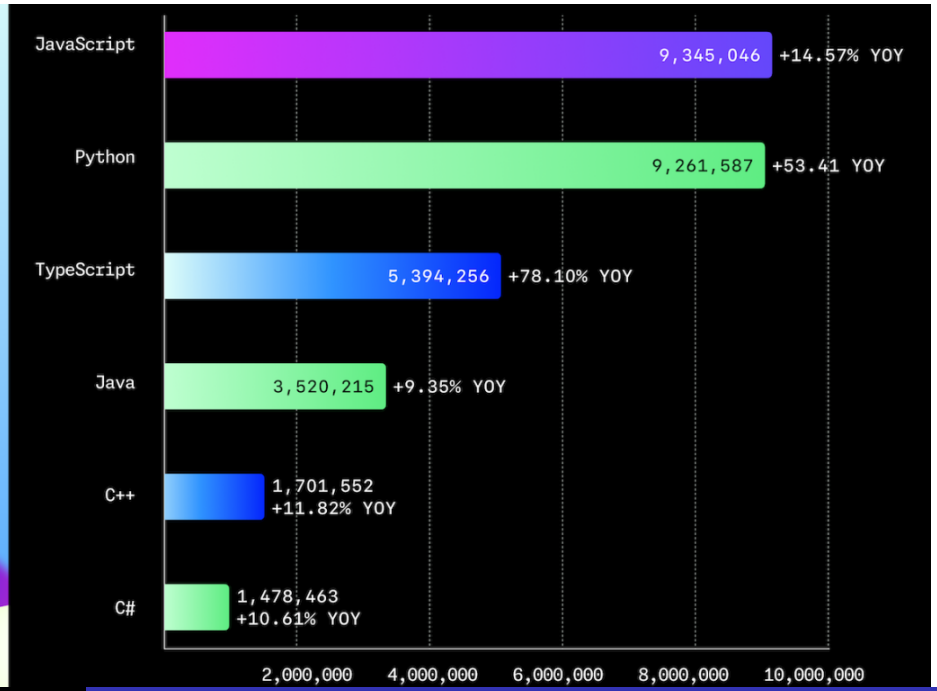
Rank	Change	Language	Share	1-year trend
1		Python	50.44 %	+20.2 %
2		Java	12.8 %	-2.0 %
3		C/C++	8.57 %	-1.2 %
4	↑	R	4.18 %	-1.3 %
5	↓	JavaScript	3.52 %	-3.4 %
6	↑↑↑↑↑↑	Objective-C	3.01 %	+0.9 %
7		PHP	2.06 %	-1.4 %
8	↑	Rust	1.86 %	-1.0 %
9	↓↓↓	C#	1.74 %	-3.0 %
10	↓↓	Swift	1.68 %	-1.2 %

It is based on the number of Google searches on the languages' tutorials.

Mostly Used Programming Languages in Github



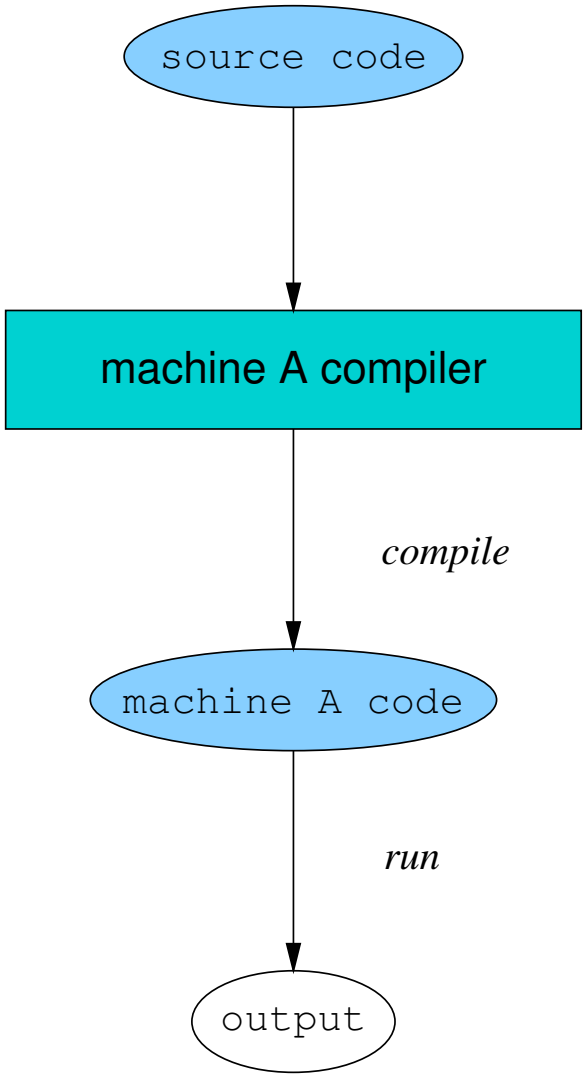




Why Learn C++ in the Age of Python?

- Python is an interpreted language that prioritizes developer speed (writing code quickly). C++ is a compiled language that prioritizes execution speed and hardware control.
- Python hides hardware complexity through automation (e.g., automatic garbage collection). C++ puts you in the driver's seat.
  - Learning C++ forces you to understand memory management (pointers, stack vs. heap) and how CPU architecture works.
  - Once you master C++, learning any other language becomes incredibly easy
- C++ is the primary choice for building foundational systems where efficiency is non-negotiable:
  - Operating Systems: Windows, macOS, and Linux kernels are written heavily in C/C++.
  - Game Engines: Industry-standard engines are built entirely on C++ to render complex 3D graphics in real-time.
  - Browsers & Database Engines: Google Chrome, MySQL, and major AI infrastructure backends rely on C++ for speed.

Compilation: From Source to Runnable Program



A compiler translates source programs into machine codes that run directly on the target computer.

For example,
a.cpp $\longrightarrow$ a.out (or a.exe).

Some C++ compilers:
gcc/g++, VC++.

- static codes
- compile once, run many
- optimized codes
  $\Rightarrow$ more efficient
- examples: FORTRAN, Pascal, C++

Programming as Problem Solving

- Understand and define the problem clearly.
  - What are the input(s) and output(s)?
  - Any constraints?
  - Which information is essential?
- Develop a solution.
  - Construct an algorithm.
- Translate the algorithm into a C++ program.
- Compile the program.
- Test the program.
- Debug the program.
- Document the program as you write the program.
- Maintain the program
  - modify the codes when conditions change.
  - enhance the codes to improve the solution.

Summary

- Why C++?
  Read the FAQ from the designer of C++, Bjarne Stroustrup.
- Which C++?
  - The language has been evolving:
    C++ 1983 $\Rightarrow$ C++ 1998 $\Rightarrow$ C++ 2003 $\Rightarrow$ C++ 2011 $\Rightarrow \cdots$
  - We will learn C++11 (but will also introduce some major new features in later version of C++ standard).
- Which compiler?
  GNU gcc/g++. It is free.
  (The compiler you will use in CSE lab is C++11-compliant.)
- Which IDE (integrated development environment) for writing programs?
  VS Code. It is free and supported by many operating systems such as Windows, Mac OS, and Linux. Other freeware like Eclipse is also fine.